# Computer Fundamentals & Problem Solving

Unit 1 Complete Syllabus: Architecture, Memory Subsystem, I/O Devices, Software Stack & Logic Development

# Module 1: Computer Systems & Architecture

Definitions, Core Characteristics, Computer Generations, Von Neumann Architecture, and CPU Component Mechanics

# Characteristics & Evolution of Computers

## ⚡Core System Characteristics

**Speed:** Executes billions of calculations per second (measured in GHz, MIPS, FLOPS).

**Accuracy:** Delivers 100% error-free output for valid input (follows GIGO: Garbage In, Garbage Out).

**Diligence:** Operates continuously without fatigue, boredom, or loss of operational focus.

**Versatility & Storage:** Performs diverse tasks ranging from medical graphics to mathematical modeling with vast data persistence.

## Computer Generations

**1st Gen (1940-56):** Vacuum Tubes | Magnetic Drums | Machine Language (ENIAC, UNIVAC).

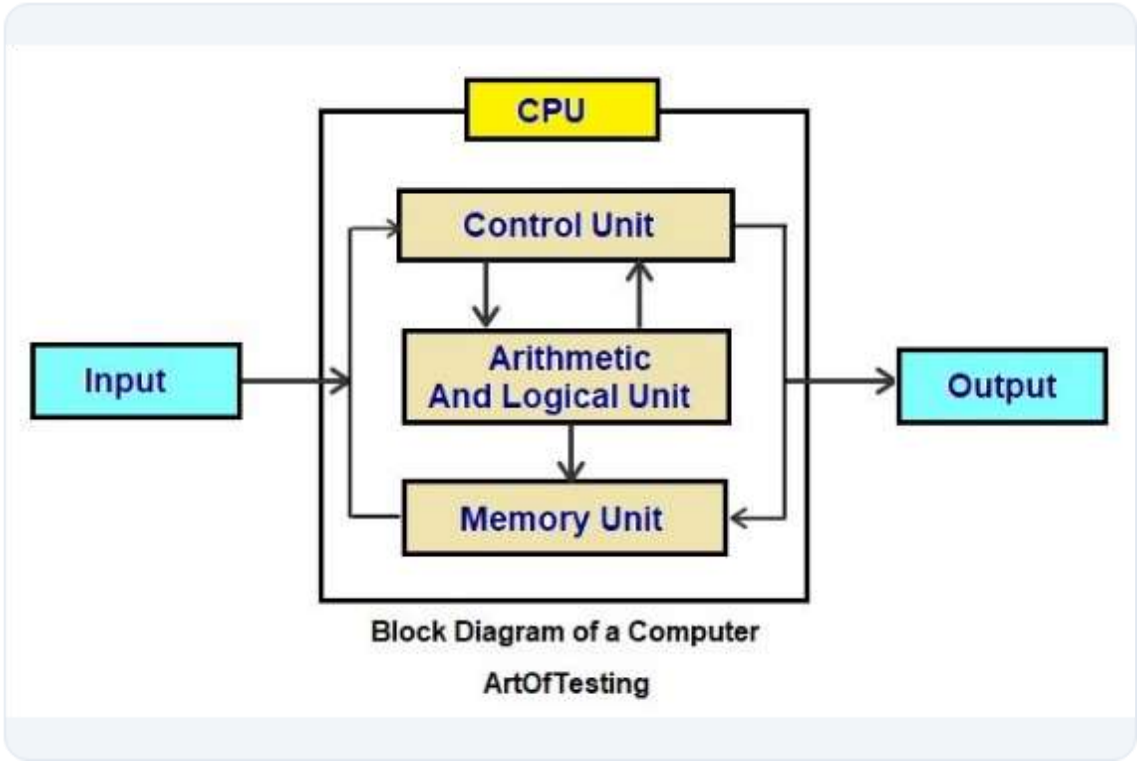
**2nd Gen (1956-63):** Transistors | Magnetic Core | Assembly Language (IBM 7090).

**3rd Gen (1964-71):** Integrated Circuits (ICs) | Operating Systems | High-Level Languages (IBM 360).

**4th & 5th Gen (1971-Present):** VLSI/ULSI Microprocessors | Parallel Processing | Artificial Intelligence.

# Block Diagram of Computer Architecture

- **Input Unit:** Converts physical human inputs or signal data into standard binary code (`0`s and `1`s) for machine processing.
- **Central Processing Unit (CPU):** The master control engine comprising the Arithmetic Logic Unit (ALU), Control Unit (CU), and internal Registers.
- **Memory Unit:** Dual-tier storage (Primary Cache/RAM and Secondary Storage) holding instructions, working data, and final results.
- **Output Unit:** Decodes computed binary representations into human-understandable visual, printed, or signal outputs.



# CPU Components & Special Registers

## Core CPU Functional Units

**Control Unit (CU):** The central orchestrator. Fetches instructions, decodes opcodes, generates control bus timing signals, and directs data flow across units.

**Arithmetic Logic Unit (ALU):** Performs basic arithmetic (+, -, *, /) and logical comparisons (<, >, ==, !=, AND, OR) required by program instructions.

## High-Speed Internal Registers

**Program Counter (PC):** Holds the memory address of the next instruction to be fetched.

**Instruction Register (IR):** Stores the current instruction being decoded and executed.

**MAR & MDR:** Memory Address Register (holds target memory address) & Memory Data Register (holds data read from/written to RAM).

**Accumulator (AC):** Holds immediate arithmetic and logic operation results.

# The Machine Instruction Cycle

## 1. Fetch Phase

The Control Unit reads the memory address stored in the **Program Counter (PC)**.

The instruction is fetched from RAM/Cache into the **Instruction Register (IR)** via the system bus.

The PC is automatically incremented (`PC = PC + 1`) to point to the subsequent instruction.

## 2. Decode Phase

The Control Unit's **Instruction Decoder** interprets the binary opcode inside the IR.

Identifies required operands and determines whether additional data needs to be loaded from memory registers into MDR.

Prepares control bus signaling paths for execution.

## 3. Execute & Store

The **ALU** carries out the requested mathematical or logical operation on loaded operands.

The final computed result is written back to the **Accumulator (AC)** or committed to main RAM.

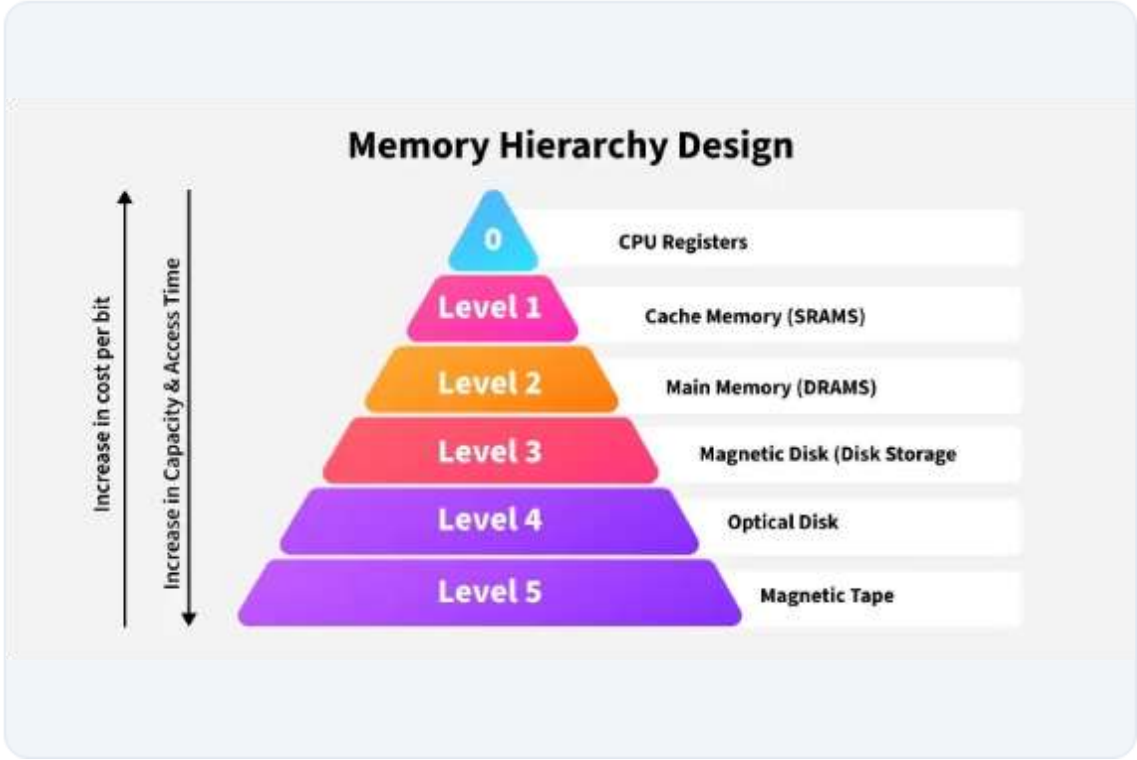
Interrupt flags are checked before restarting the next fetch cycle.

# Module 2: Memory Subsystems & Storage Hierarchy

Primary vs Secondary Storage, RAM vs ROM, Cache Memory Levels, and Storage Technologies

# The Computer Memory Hierarchy

- **CPU Registers:** Smallest capacity (Bytes), fastest access time (~0.5 ns), located directly inside the CPU core.
- **Cache Memory (L1, L2, L3):** High-speed SRAM placed between CPU and RAM to bridge CPU-memory speed gap.
- **Primary Memory (RAM):** Volatile DRAM holding currently active OS programs and runtime variables (~10-50 ns).
- **Secondary Storage (SSD/HDD):** Non-volatile, high-capacity persistent storage for permanent files and applications.



# Primary Memory: RAM vs ROM

## RAM (Random Access Memory)

**Volatility:** Volatile (loses all content immediately when power is turned off).

**Function:** Provides temporary read/write workspace for active programs.

**SRAM vs DRAM:**

- *SRAM (Static RAM):* Uses flip-flops, faster, expensive, used in Cache.
- *DRAM (Dynamic RAM):* Uses capacitors, requires periodic refresh, higher density, used in Main Memory.

## ROM (Read Only Memory)

**Volatility:** Non-volatile (retains data permanently across reboots).

**Function:** Contains essential boot code (BIOS / UEFI firmware / Bootloader).

**Types of ROM:**

- *PROM:* Programmable once by manufacturer/user.
- *EPROM:* Erasable using ultraviolet light.
- *EEPROM:* Electrically Erasable (Flash ROM, standard today).

# Secondary Storage Technologies

| Storage Type | Underlying Technology | Typical Read/Write Speed | Key Advantages / Disadvantages |
|---|---|---|---|
| **Hard Disk Drive (HDD)** | Magnetic platters with spinning mechanical read/write head | 80 MB/s - 160 MB/s | Very low cost per GB; vulnerable to physical shock, higher power consumption. |
| **Solid State Drive (SSD)** | NAND Flash Semiconductor Floating-gate Memory Chips | 500 MB/s - 7000 MB/s (NVMe) | Extremely high speed, zero noise, highly durable; higher cost per GB. |
| **Optical Storage (CD/DVD/BD)** | Laser optics reading reflective pits and lands on disc surface | 10 MB/s - 50 MB/s | Inexpensive distribution medium; low capacity, easily scratched, degrading. |
| **Magnetic Tape** | Sequential access magnetic strip reel | 100 MB/s - 300 MB/s (Sequential) | Massive archival capacity (PB level), ultra-low cost; slow random access. |

# Module 3: Peripheral Hardware & I/O Systems

Comprehensive Categorization of Input Devices, Output Devices, and Printing Technologies

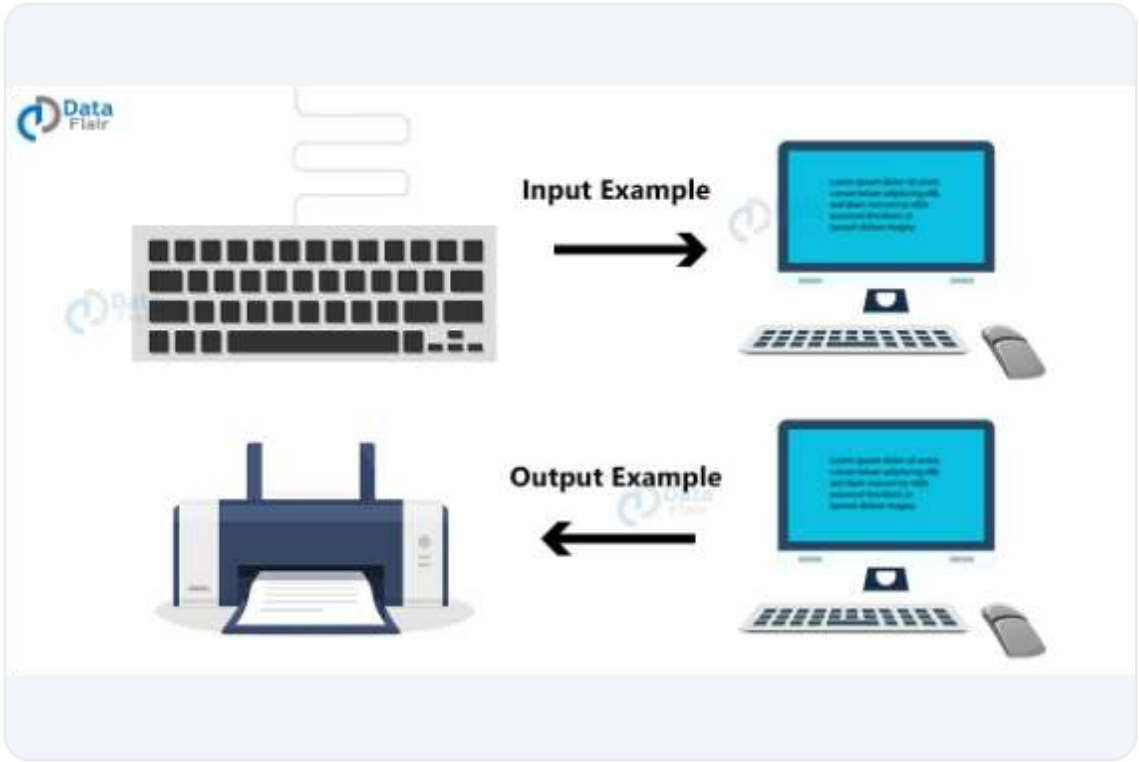
# Input and Output Hardware Systems

**Specialized Input Devices:**

- *OCR (Optical Character Recognition):* Scans printed text into editable text.
- *MICR (Magnetic Ink Character Recognition):* Validates bank cheque numbers.
- *OMR (Optical Mark Reader):* Evaluates multiple-choice exam answer sheets.

**Output Printers (Impact vs Non-Impact):**

- *Impact Printers:* Mechanical striking (Dot Matrix, Daisy Wheel). Loud, slow.
- *Non-Impact Printers:* Laser & Inkjet technology. High resolution, quiet, fast.

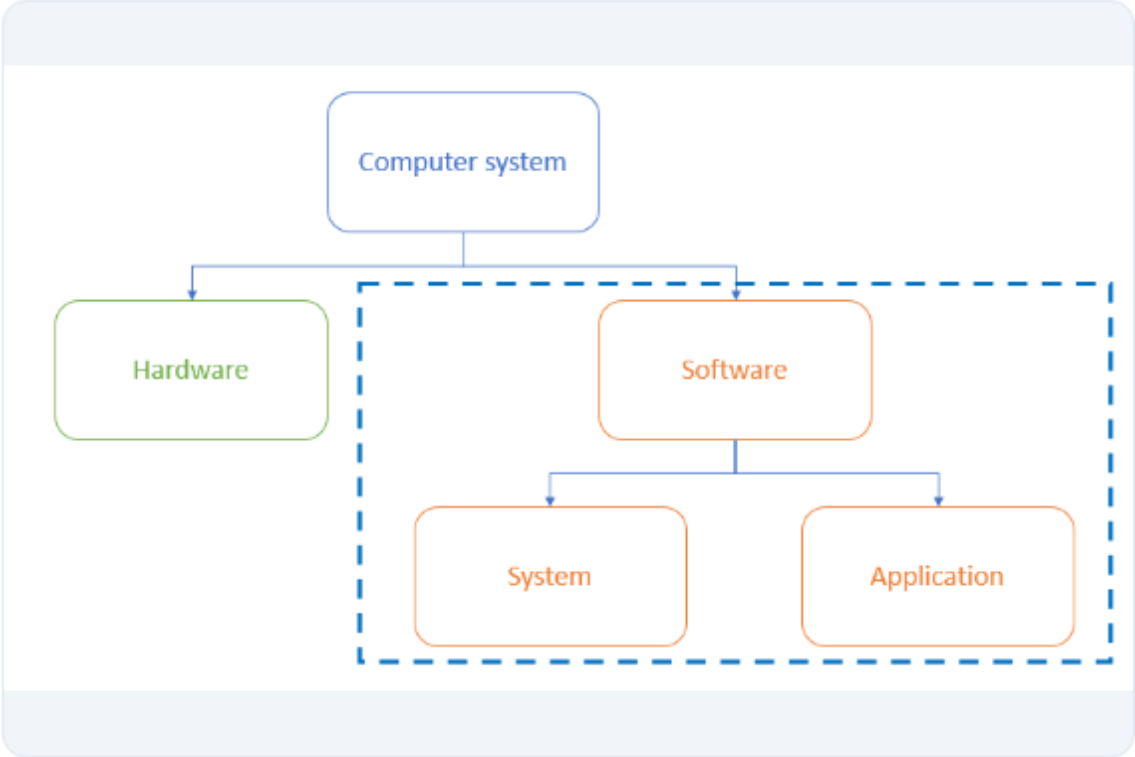


# Module 4: Software Stack & Language Translators

System Software vs Application Software, Operating Systems, Compilers, Interpreters, and Linkers

# Software Classification Architecture

- **System Software:** Controls hardware interactions and provides platform execution environments (e.g., Windows, Linux Kernel, OS drivers).
- **Application Software:** Performs end-user specific tasks (e.g., Web Browsers, CAD software, Word Processors, IDEs).
- **Utility Software:** Maintenance tools that optimize disk space, security, and hardware health (e.g., Antivirus, Defrag, Compression tools).
- **Firmware:** Low-level software hardcoded into non-volatile EEPROM/Flash chips to boot hardware.



# Language Translators & Execution Build Pipeline

## </>Compiler vs Interpreter vs Assembler

**Compiler:** Translates entire High-Level code into Machine Code (.obj/.exe) in one pass before execution (e.g., C, C++, Rust). Fast runtime speed.

**Interpreter:** Translates and executes High-Level source code line-by-line directly at runtime (e.g., Python, JavaScript). Slower runtime speed.

**Assembler:** Converts Low-Level Assembly mnemonic code (ADD, MOV, SUB) directly into native binary machine code.

## Linker & Loader Mechanics

**Linker:** Combines multiple compiled object code modules (`.obj`) with external pre-compiled library files to produce a single unified executable file (`.exe`).

**Loader:** Operating system utility that loads the executable program from secondary storage (HDD/SSD) into primary memory (RAM), sets up stack/heap memory, and initializes execution.

# Module 5: Problem Solving Logic - Algorithms & Flowcharts

Algorithm Principles, Characteristics, Flowchart Standards, and Program Execution Logic

# Algorithm Principles & Flowchart Symbols

**Core Algorithm Criteria:**

- *Finiteness:* Must terminate after a finite number of steps.
- *Definiteness:* Each step must be clear and unambiguous.
- *Input/Output:* Accepts 0+ inputs and produces 1+ outputs.
- *Effectiveness:* Operations must be basic and feasible.

**Standard Flowchart Symbols:**

- *Oval:* Start / End Terminal.
- *Parallelogram:* Input / Output operations.
- *Rectangle:* Processing / Calculations.
- *Diamond:* Decision making / Condition evaluation.

| Symbol | Name | Function |
|---|---|---|
|  | Start/end | An oval represents a start or end point |
|  | Arrows | A line is a connector that shows relationships between the representative shapes |
|  | Input/Output | A parallelogram represents input or output |
|  | Process | A rectangle represents a process |
|  | Decision | A diamond indicates a decision |

# Algorithm & Code Examples in C

## ⇄Example 1: Swapping Without 3rd Var

```c
#include

int main() {
    int a = 5, b = 10;
    // Swapping mathematical logic
    a = a + b; // a = 15
    b = a - b; // b = 5
    a = a - b; // a = 10

    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

## 🏆Example 2: Largest of 3 Numbers

```c
#include

int main() {
    int x = 25, y = 40, z = 15;
    if (x >= y && x >= z)
        printf("Largest: %d", x);
    else if (y >= x && y >= z)
        printf("Largest: %d", y);
    else
        printf("Largest: %d", z);
    return 0;
}
```